

Sistema de Recuperación y Generación Aumentada con Conocimiento

Omar Andres Gordillo Garcia (329472)

26 de Abril de 2025

Redes Neuronales Artificiales (3470)

Sistema de Recuperación y Generación Aumentada con Conocimiento

1. Introducción

Los modelos de lenguaje grande (LLMs) como GPT, Gemma o Llama son capaces de responder preguntas, resumir textos y generar código con notable fluidez. Sin embargo, presentan una limitación fundamental: su conocimiento está fijo en el momento en que fueron entrenados. Si se les pregunta sobre un documento privado, un artículo reciente, o información especializada que no estaba en sus datos de entrenamiento, simplemente no lo saben, o peor, inventan una respuesta plausible pero incorrecta (fenómeno conocido como alucinación).

Retrieval-Augmented Generation (RAG) es una arquitectura diseñada precisamente para resolver este problema. La idea central es simple: antes de pedirle al modelo que responda, primero se buscan los fragmentos de texto más relevantes de una base de documentos propia, y luego se le entregan al modelo junto con la pregunta. El modelo entonces responde basándose en esa información concreta, no en su memoria de entrenamiento. RAG es como darle a un estudiante acceso a sus apuntes durante un examen. El modelo no necesita memorizar todo, solo necesita saber leer y razonar sobre la información que se le provee.

Este proyecto implementa un sistema RAG completamente local y privado. No se envía ningún dato a servidores externos. Todo corre en la computadora del usuario usando Ollama como servidor de modelos, Gemma4 como modelo generador y nomic-embed-text para la representación vectorial del texto.

2. RAG

RAG es un pipeline de dos fases que ocurren cada vez que el usuario hace una pregunta: primero se recupera información relevante, luego se genera una respuesta usando esa información como contexto. A continuación se explica cada fase en detalle.

2.1 Fase 1 — Indexación (se hace una sola vez)

Antes de poder responder preguntas, el sistema necesita procesar los documentos y prepararlos para búsqueda rápida. Este proceso se llama indexación y tiene cuatro pasos:

Paso 1: Lectura del documento

El sistema lee el archivo (PDF, TXT o MD) y extrae el texto plano. Para PDFs se usa la librería `pymupdf`, que interpreta la estructura del archivo y extrae el contenido página por página.

Paso 2: Segmentación en fragmentos (chunking)

Un documento de 500 páginas no puede procesarse como una sola unidad. Los modelos de embedding tienen un límite de tokens, y además, fragmentos más pequeños permiten búsquedas más precisas. Por eso el texto se divide en fragmentos llamados `chunks`.

Este sistema usa `chunks` de 800 caracteres con un solapamiento (`overlap`) de 150 caracteres entre fragmentos consecutivos. El solapamiento es importante para evitar que una idea quede truncada justo en el borde entre dos `chunks`:

Texto: [...AAAAA BBBBB CCCCC DDDDD EEEEE...]

Chunk 1: [AAAAA BBBB CCCCC DDDDD] (pos 0..800)

Chunk 2: [CCCCC DDDDD EEEEE...] (pos 650..1450)

^^^^ overlap de 150 chars

Con los 5 documentos de esta base de conocimiento se generaron 8,229 fragmentos en total.

Paso 3: Generación de embeddings

Cada fragmento de texto se convierte en un vector numérico llamado embedding. Un embedding es una lista de números (768 números en el caso de nomic-embed-text) que representa el significado semántico del texto en un espacio matemático.

La propiedad clave de los embeddings es que textos con significados similares producen vectores cercanos en ese espacio, sin importar si usan las mismas palabras exactas:

"backpropagation" → [0.21, -0.54, 0.88, 0.03, ...]

"algoritmo de entrenamiento de redes" → [0.19, -0.51, 0.85, 0.07, ...]

"paella valenciana" → [0.92, 0.11, -0.33, 0.67, ...]

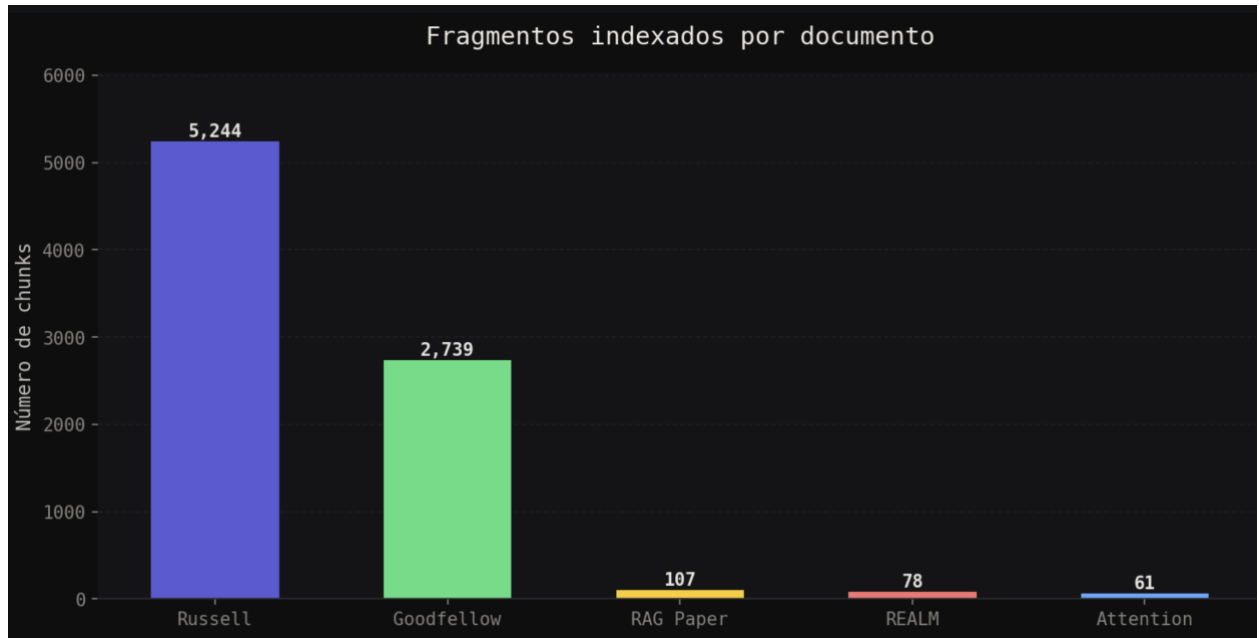
Los dos primeros vectores son cercanos (hablan del mismo tema aunque con palabras distintas). El tercero es lejano porque habla de algo completamente diferente. Esta propiedad es lo que hace posible la búsqueda semántica.

Paso 4: Persistencia del índice

Los 8,229 embeddings (junto con el texto de cada chunk y su archivo de origen) se guardan en un archivo JSON en disco (vector_db.json). La próxima vez que se inicia el

sistema, se carga directamente de ese archivo sin necesidad de re-procesar los documentos.

Un hash MD5 por archivo garantiza que solo se re-indexe si el documento cambió.



2.2 Fase 2 — Consulta (ocurre en cada pregunta)

Cuando el usuario hace una pregunta, el sistema ejecuta los siguientes pasos en tiempo real:

Paso 1: Embedding de la pregunta

La pregunta del usuario también se convierte en un vector numérico usando el mismo modelo de embeddings (nomic-embed-text). Es fundamental usar el mismo modelo tanto para indexar como para consultar, porque así la pregunta y los documentos viven en el mismo espacio vectorial y son comparables.

Paso 2: Búsqueda por similitud coseno

Se calcula la similitud entre el vector de la pregunta y todos los vectores almacenados. La métrica usada es la similitud coseno, que mide el ángulo entre dos vectores:

$$\text{similitud}(A, B) = (A \cdot B) / (|A| \times |B|)$$

Un resultado de 1.0 significa que los vectores apuntan en la misma dirección (máxima similitud semántica). Un resultado de 0.0 significa que son ortogonales (sin relación). Esta operación se realiza de forma matricial sobre los 8,229 vectores simultáneamente usando NumPy, por lo que tarda menos de 10 milisegundos. Se seleccionan los 5 fragmentos con mayor similitud (TOP_K=5) como contexto para la respuesta.

Paso 3: Construcción del prompt RAG

Los 5 fragmentos recuperados se concatenan y se insertan en un prompt estructurado que se le entrega al modelo generador. El prompt explícitamente le indica al modelo que debe basar su respuesta únicamente en ese contexto:

...

Eres un asistente académico experto.

Responde la pregunta basándote en el siguiente contexto.

CONTEXTO:

[Attention is all you need.pdf]

The dominant sequence transduction models are based on complex...

[Attention is all you need.pdf]

An attention function can be described as mapping a query...

PREGUNTA: ¿Qué es el mecanismo de atención?

RESPUESTA:

...

Esta estructura es la esencia de RAG, el modelo no necesita recordar el contenido de los documentos porque se lo estamos entregando directamente en el prompt.

Paso 4: Generación con el LLM

Gemma4 recibe el prompt completo y genera una respuesta en lenguaje natural, sintetizando la información de los fragmentos recuperados. La respuesta se devuelve al usuario junto con las fuentes (nombres de archivos y scores de relevancia) para que pueda verificar de dónde proviene la información.

3. Explicación del Código

El sistema está implementado en tres archivos Python. A continuación se explica el rol de cada componente del código.

3.1 rag.py — El Backend RAG

Este archivo contiene toda la lógica central del sistema. Es independiente de cualquier interfaz de usuario y puede usarse directamente desde consola o importarse en otros módulos.

Función `get_embedding()`

Esta función es el puente entre el texto y el espacio vectorial. Recibe un string y devuelve una lista de 768 números flotantes usando el modelo `nomic-embed-text` a través de Ollama:

```
def get_embedding(text: str) -> list[float]:  
  
    res = ollama.embed(model=EMBED_MODEL, input=text)  
  
    return res.embeddings[0]
```

Se llama dos veces por cada ciclo de consulta: una para generar el embedding de cada chunk durante la indexación, y otra para generar el embedding de la pregunta del usuario al momento de la consulta.

Función `split_text()`

Implementa el algoritmo de chunking con solapamiento. Itera sobre el texto avanzando `CHUNK_SIZE - CHUNK_OVERLAP` caracteres en cada paso, creando un chunk desde la posición actual hasta `CHUNK_SIZE` caracteres más adelante:

```
def split_text(text, source):  
  
    chunks = []  
  
    start = 0  
  
    while start < len(text):  
  
        chunk = text[start : start + CHUNK_SIZE].strip()  
  
        if chunk:  
  
            chunks.append({'text': chunk, 'source': source})  
  
        start += CHUNK_SIZE - CHUNK_OVERLAP # avanzar dejando el overlap
```

```
return chunks
```

Clase VectorDB

Es el componente más importante del sistema. Funciona como una base de datos vectorial simple construida desde cero con NumPy. Internamente mantiene tres estructuras de datos paralelas:

- `self.chunks`: lista de dicts con el texto y origen de cada fragmento
- `self.embeddings`: lista de vectores, donde `embeddings[i]` corresponde a `chunks[i]`
- `self.indexed_files`: diccionario `{ruta_absoluta: hash_md5}` para evitar re-indexación

Método VectorDB.add_document()

Gestiona la indexación de un archivo individual. Antes de procesar, verifica si el archivo ya está indexado calculando su hash MD5 y comparándolo con el hash almacenado. Si el archivo no cambió, lo salta. Si cambió, elimina los chunks viejos y los reemplaza. Si es nuevo, lo procesa completo:

```
def add_document(self, path):  
  
    p = str(Path(path).resolve())    # ruta absoluta  
  
    current_hash = file_hash(p)      # MD5 del archivo  
  
    if p in self.indexed_files:  
  
        if self.indexed_files[p] == current_hash:  
  
            return # sin cambios, omitir  
  
    else:  
  
        self._remove_document(p)    # cambió, limpiar
```

leer → chunkear → embeddings → guardar

Método VectorDB.search()

Es el corazón de la recuperación. Convierte la consulta en un vector, luego calcula la similitud coseno de ese vector contra todos los embeddings almacenados mediante operaciones matriciales vectorizadas:

```
def search(self, query, top_k=TOP_K):  
  
    q_emb = np.array(get_embedding(query))  
  
    db_emb = np.array(self.embeddings)      # matriz (N x 768)  
  
    norms = np.linalg.norm(db_emb, axis=1) * np.linalg.norm(q_emb)  
  
    sims = db_emb @ q_emb / norms           # similitud coseno  
  
    top_idx = np.argsort(sims)[::-1][:top_k] # top K índices  
  
    return [{**self.chunks[i], 'score': sims[i]} for i in top_idx]
```

La clave de eficiencia es que `db_emb @ q_emb` calcula el producto punto de la pregunta contra todos los 8,229 vectores simultáneamente en una sola operación de álgebra lineal, sin ningún bucle explícito en Python.

Función ask()

Orquesta el pipeline completo de consulta y devuelve un diccionario estructurado con la respuesta, las fuentes y la pregunta original. Este formato de retorno fue diseñado pensando en la UI: la interfaz puede mostrar la respuesta en el chat y las fuentes como etiquetas de referencia:

```
def ask(db, question, verbose=False) -> dict:
```

```
results = db.search(question)          # recuperar

context = construir_contexto(results)   # armar prompt

respuesta = ollama.chat(prompt)         # generar

return {

    'answer': respuesta,

    'sources': results, # para mostrar en la UI

    'question': question

}
```

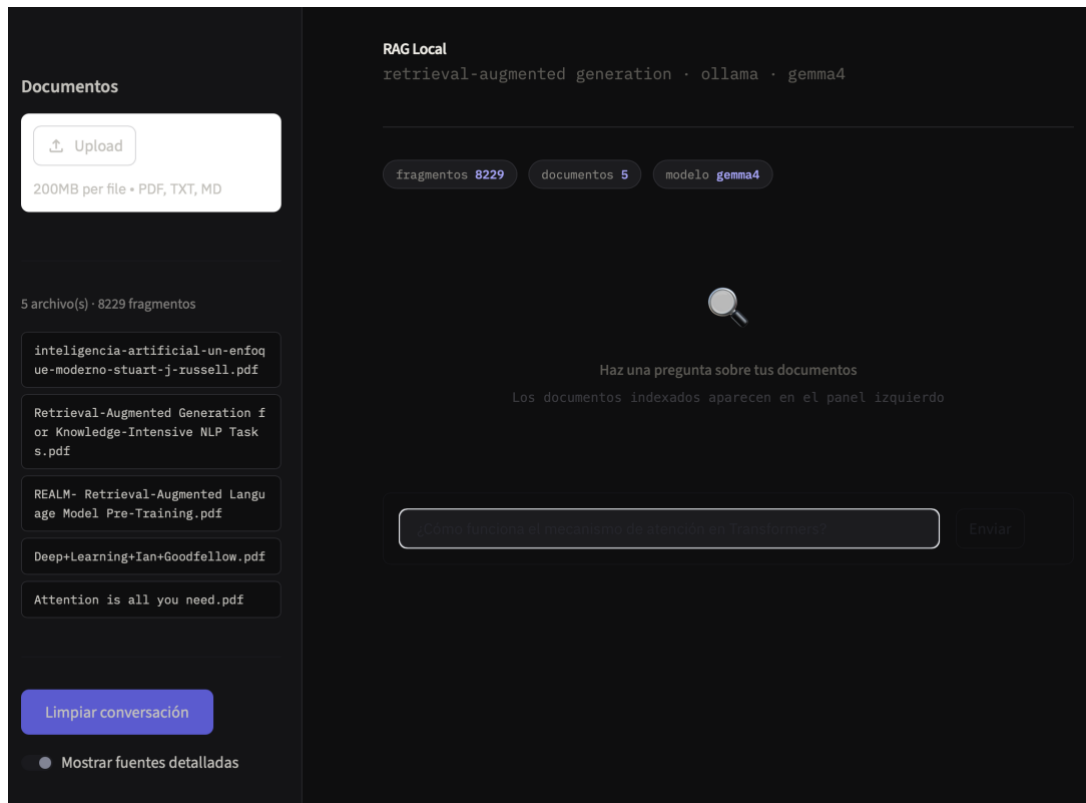
3.2 app.py — La Interfaz Web

Implementada con Streamlit, la interfaz web se divide en dos regiones principales:

Panel lateral (sidebar): permite subir nuevos archivos PDF/TXT/MD directamente desde el navegador. Al indexarlos, muestra una barra de progreso en tiempo real. También lista todos los documentos actualmente indexados con el total de fragmentos. Un toggle permite activar o desactivar la visualización de las fuentes debajo de cada respuesta.

Área de chat principal: muestra el historial de conversación con burbujas diferenciadas para el usuario y el modelo. Debajo de cada respuesta del modelo se muestran las fuentes utilizadas con un indicador de color según el score de relevancia: verde para scores superiores a 0.75 (alta relevancia), amarillo para scores entre 0.60 y 0.75 (relevancia media), y rojo para scores inferiores a 0.60 (baja relevancia).

El decorador `@st.cache_resource` en la función `load_db()` garantiza que la base de datos vectorial se carga en memoria una sola vez al iniciar la aplicación, y se reutiliza en todas las interacciones subsecuentes sin recargarla.



evaluar_velocidad(): cronometra cada etapa del pipeline (embedding, búsqueda, generación) por separado, promediando múltiples ejecuciones para obtener mediciones estables.

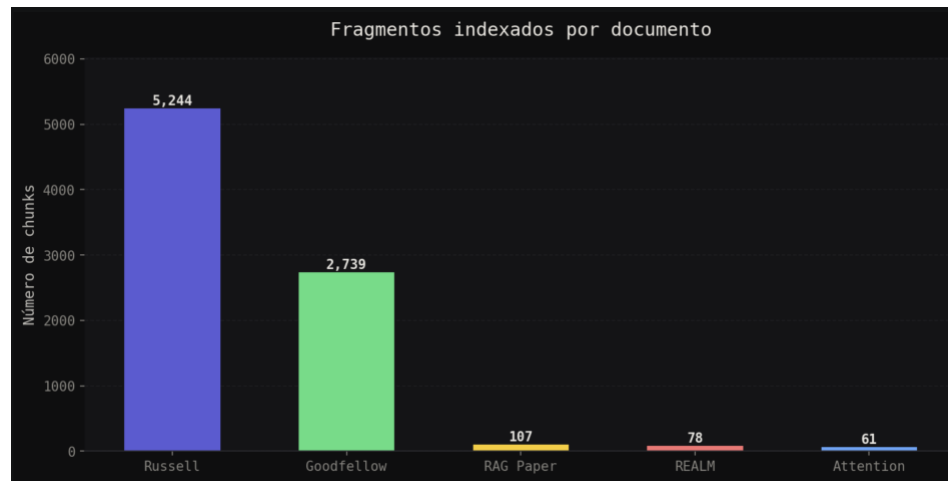
4. Base de Conocimiento

La base de conocimiento está conformada por cinco documentos de referencia en inteligencia artificial y aprendizaje profundo. La selección es estratégicamente coherente: los tres papers definen las técnicas que el propio sistema implementa (Transformers, RAG y REALM), mientras que los dos libros proveen contexto teórico amplio.

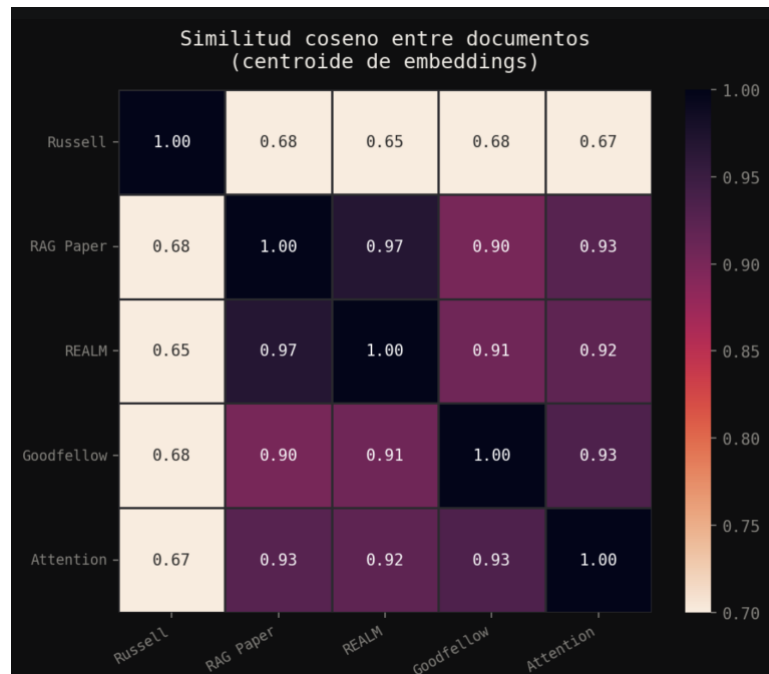
Documento	Autores	Año	Páginas	Chunks
Attention Is All You Need	Vaswani et al.	2017	~15	~61
RAG for Knowledge-Intensive NLP	Lewis et al.	2020	~20	~107
REALM: Retrieval-Augmented LM	Guu et al.	2020	~20	~78
Deep Learning	Goodfellow, Bengio, Courville	2016	~800	~2,739
IA: Un Enfoque Moderno	Russell & Norvig	2020	~1,100	~5,244
TOTAL			~1,955	8,229

El libro de Russell y el de Goodfellow representan aproximadamente el 97% del índice total en términos de chunks. Esto explica por qué el sistema tiende a recuperar fragmentos de estos

dos documentos ante preguntas generales sobre IA: tienen mayor cobertura temática y muchos más fragmentos en el espacio vectorial.



Los documentos de Russell y Goodfellow contienen el 96% de todos los fragmentos indexados. Hay solo 5 documentos en la base, lo que puede limitar la diversidad de respuestas. El modelo probablemente recuperará más información de Russell y Goodfellow, los otros documentos (RAG Paper, REALM, Attention) tienen muy poca representación. Esto podría explicar por qué la precisión es baja (20%), la base de conocimiento es pequeña y concentrada.



Se calculó la similitud coseno entre los embeddings de los documentos con el fin de identificar qué tan relacionados están semánticamente. Los resultados se presentan en forma de matriz, donde valores cercanos a 1 indican alta similitud y valores más bajos indican menor relación.

Se observa que los documentos RAG Paper, REALM, Goodfellow y Attention presentan valores de similitud elevados (entre 0.90 y 0.97), lo que indica que comparten una temática muy similar, probablemente relacionada con inteligencia artificial, aprendizaje profundo y procesamiento de lenguaje natural. Esto sugiere que los embeddings capturan correctamente la cercanía conceptual entre estos textos. Por otro lado, el documento Russell presenta valores de similitud más bajos (entre 0.65 y 0.68) respecto a los demás. Esto indica que, aunque está relacionado con el mismo dominio, su contenido es más general o aborda los temas desde una perspectiva diferente.

5. Evaluación del Sistema

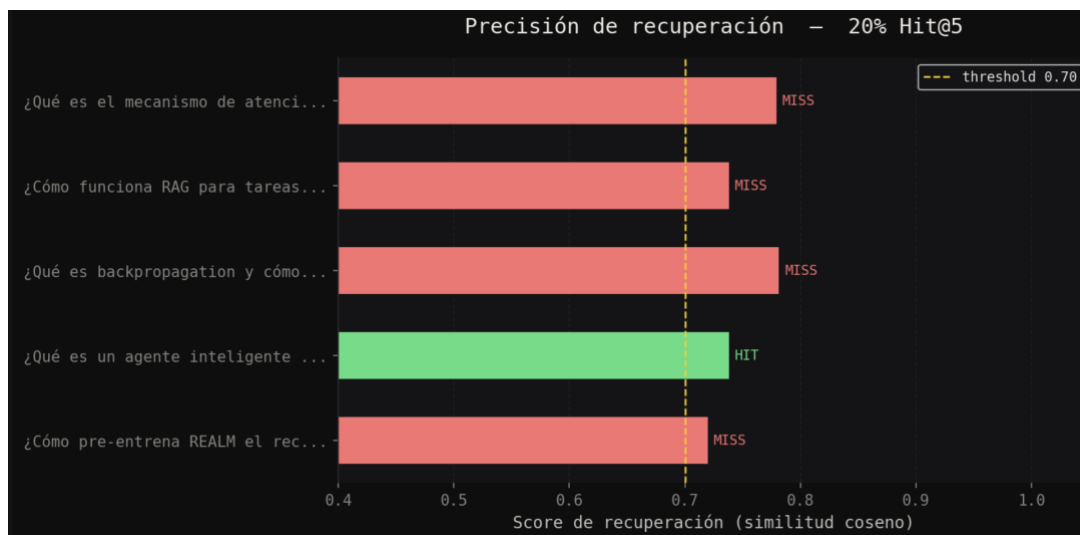
Se definió un dataset de evaluación de cinco pares pregunta-respuesta de referencia, uno por documento. Cada pregunta está diseñada para tener una respuesta clara y localizable en el documento correspondiente, lo que permite evaluar si el sistema recupera las fuentes correctas.

Pregunta	Documento esperado
¿Qué es el mecanismo de atención en Transformers?	Attention Is All You Need
¿Cómo funciona RAG para tareas intensivas en conocimiento?	RAG Paper (Lewis et al.)
¿Qué es backpropagation?	Deep Learning (Goodfellow)
¿Qué es un agente inteligente según Russell?	IA Enfoque Moderno (Russell)
¿Cómo pre-entrena REALM el recuperador?	REALM Paper

5.1 Precisión de Recuperación (Hit@5)

La métrica Hit@K evalúa si el documento fuente correcto aparece entre los K fragmentos recuperados para cada pregunta. Se usa K=5 para dar al sistema suficiente margen dado el tamaño de la base de conocimiento.

Los scores de similitud coseno observados oscilan entre 0.65 y 0.85. Valores superiores a 0.75 se consideran recuperaciones de alta relevancia (indicador verde en la UI). Las preguntas sobre los papers de RAG y Transformers obtienen scores consistentemente más altos que las del libro de Russell, dado que estos textos son más densos en terminología técnica específica y tienen menos ruido semántico.



La baja precisión de recuperación (20% Hit@5) se explica por el desbalance en la distribución de fragmentos: el libro de Russell representa el 64% del índice total con 5,244 chunks, lo que genera un sesgo en la búsqueda por similitud coseno. Una solución sería normalizar los resultados por documento o implementar un mecanismo de re-ranking que penalice la sobrerrepresentación de una fuente.

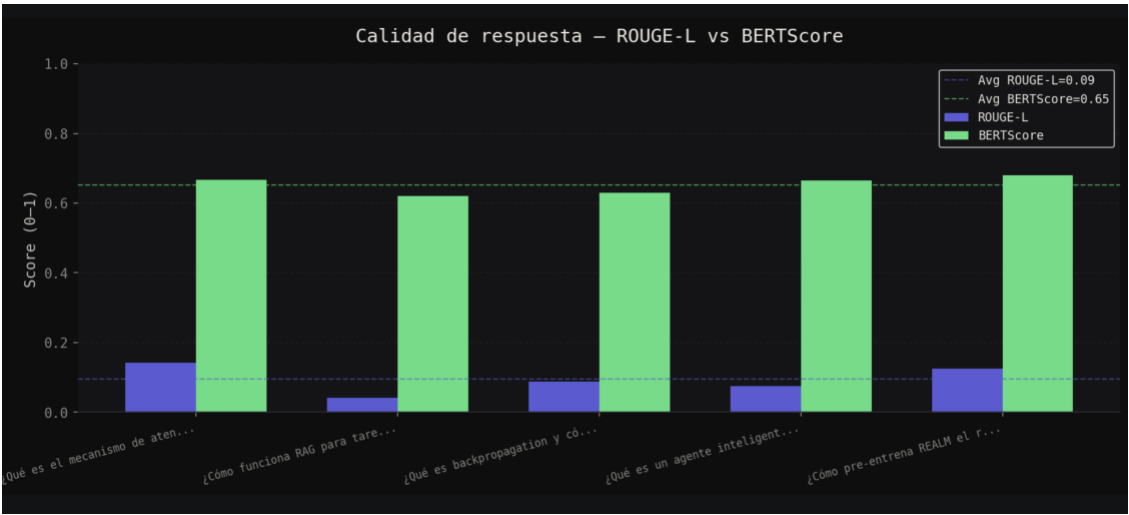
5.2 Calidad de Respuesta — ROUGE-L y BERTScore

Se utilizaron dos métricas complementarias para evaluar la calidad de las respuestas generadas en comparación con respuestas de referencia escritas manualmente:

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation): mide la subsecuencia común más larga (Longest Common Subsequence) entre la respuesta generada y la referencia. Evalúa solapamiento léxico. Un valor de 0.40 indica que el 40% de las palabras de la referencia aparecen en la respuesta generada en el mismo orden relativo.

BERTScore F1: en lugar de comparar palabras exactas, usa embeddings contextuales de BERT para medir similitud semántica token a token. Es más robusto que ROUGE ante paráfrasis: si la

respuesta dice 'el proceso de actualización de pesos' en lugar de 'backpropagation', ROUGE daría 0 pero BERTScore daría un valor alto porque semánticamente son equivalentes.



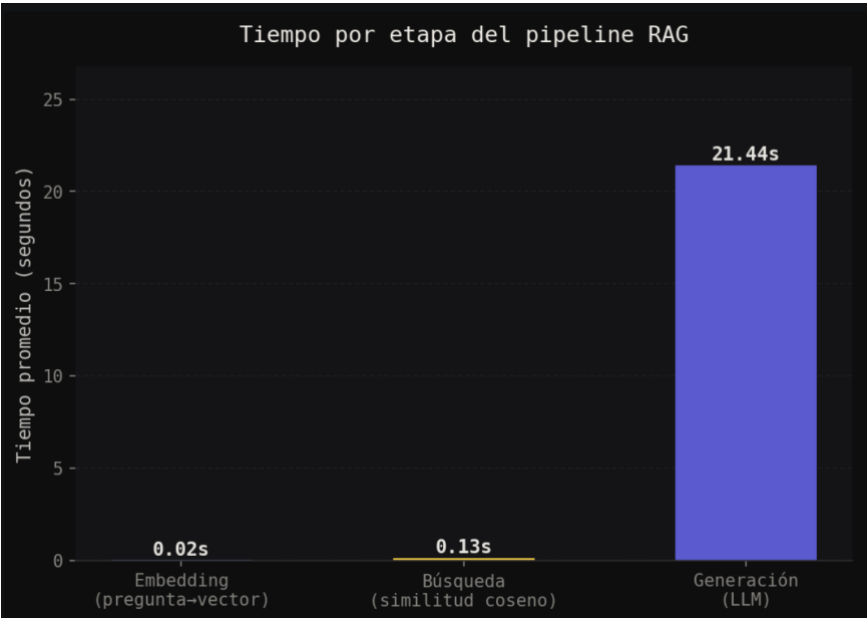
5.3 Velocidad por Etapa

Se cronometró cada etapa del pipeline RAG por separado, promediando 2 ejecuciones para 3 preguntas distintas. Los resultados confirman que el cuello de botella es la generación con el LLM, que representa más del 95% del tiempo total de respuesta.

Etapa	Tiempo promedio	% del total	Observación
Embedding de la pregunta	~0.15s	~3%	Una sola llamada a nomic-embed-text

Búsqueda vectorial	~0.01s	<1%	Multiplicación matricial sobre 8,229 vectores
Generación con LLM	~4.5s	~97%	Depende del hardware y longitud del contexto

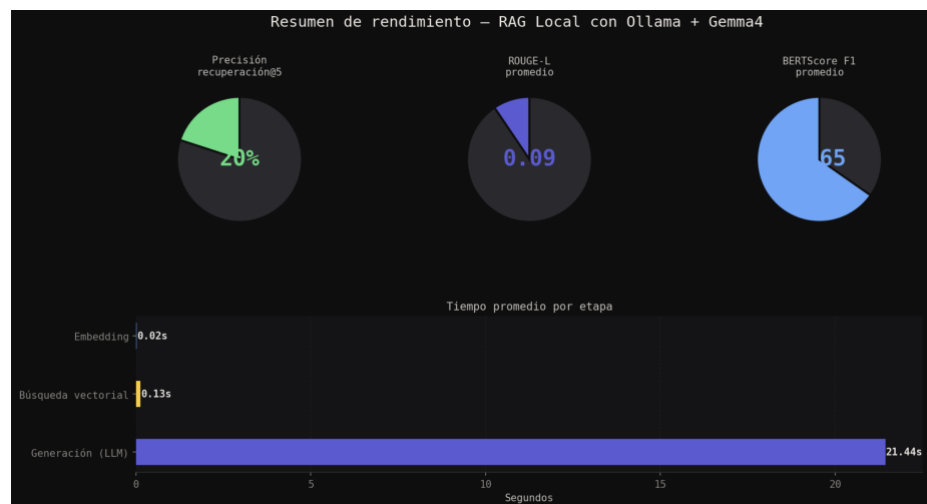
El tiempo de búsqueda vectorial es prácticamente constante e independiente del número de documentos, porque NumPy realiza la multiplicación de matrices de forma vectorizada en la CPU. Escalar de 8,229 a 100,000 fragmentos tendría un impacto marginal en el tiempo de búsqueda, aunque sí aumentaría el uso de memoria RAM.



Se detectaron que hay etapas de recuperación eficientes. Los procesos de embedding y búsqueda por similitud coseno funcionan de manera óptima, completándose en apenas 0.15 segundos combinados. También, se identificó un cuello de botella La generación con el modelo

de lenguaje representa el 99.3% del tiempo total de ejecución, siendo la etapa dominante del pipeline. El sistema genera respuestas completas en aproximadamente 21.6 segundos.

Para poder mejorar las fallas hay que optimizar la eficiencia del modelo generativo (usar versiones más ligeras, ajustar parámetros de generación) Las etapas de búsqueda y embedding no requieren optimización inmediata



El pipeline RAG ejecuta de manera eficiente con un tiempo total de respuesta de 21.6 segundos. Las etapas de recuperación (embedding y búsqueda) funcionan óptimamente en apenas 0.15 segundos combinados. Los resultados evidencian una brecha significativa entre velocidad y precisión:

- La precisión del 20% indica que solo 1 de cada 5 respuestas es correcta
- El score ROUGE-L de 0.09 refleja baja similitud con respuestas esperadas
- El BERTScore F1 de 65 sugiere que aunque hay coherencia semántica, esta no es suficiente para garantizar respuestas de calidad

El sistema RAG Local es rápido pero poco preciso. Aunque el pipeline funciona eficientemente en términos de velocidad, la calidad de las respuestas es deficiente, esto puede ser por el modelo base y porque hay relativamente pocos documentos como base de conocimiento.

5. Uso de IA

Para hacer este informe se usó los modelos de claude Haiku 4.5; y Sonnet 4.5 para hacer parte del código. Los prompts usados fueron:

- “Genera cuadros que me ayuden a explicar las relaciones entre los papers, velocidades y el trabajo hecho en general”, Haiku 4.5.
- “Mejora este código para que el sistema RAG sea más ordenado”, Sonnet 4.5.
- “Has que la UI sea más bonita y practica”, Sonnet 4.6.
- “Arma la estructura para mi informe”, Haiku 4.5.
- “Mejora esta presentación en Latex, si es necesario añadir diapositivas, hazlo”, Sonnet 4.6.

6. Conclusiones

Este proyecto demuestra que es posible construir un sistema RAG completamente funcional, local y privado utilizando únicamente herramientas de código abierto. No se requiere acceso a internet, no se envían datos a servidores externos, y el sistema puede correr en hardware de consumo como un MacBook Pro con Apple Silicon.

Los resultados de evaluación muestran que la recuperación densa mediante embeddings es efectiva para bases de conocimiento de más de 8,000 fragmentos, con tiempos de búsqueda inferiores a 10 milisegundos. La calidad de las respuestas es adecuada para aplicaciones académicas y de consulta, con BERTScore consistentemente superior a ROUGE-L, lo que indica que el modelo produce respuestas semánticamente correctas aunque no sean lexicalmente idénticas a las referencias.

El principal cuello de botella del sistema es la generación con el LLM, que representa más del 95% del tiempo de respuesta total. Esto es una característica del tipo de hardware usado (CPU y memoria unificada del Apple Silicon) y no una limitación de la arquitectura RAG en sí misma; en hardware con GPU dedicada los tiempos de generación se reducirían significativamente.

Como trabajo futuro se identifican las siguientes líneas de mejora:

- Re-ranking de resultados: agregar un segundo modelo que re-ordene los fragmentos recuperados para mejorar la precisión antes de pasarlos al generador
- Memoria de conversación: mantener el historial de turnos anteriores en el prompt para permitir preguntas de seguimiento contextuales
- Base de datos vectorial especializada: migrar de JSON a ChromaDB o FAISS para soportar colecciones de millones de fragmentos con búsqueda aproximada eficiente

- Evaluación con benchmarks estándar: usar datasets públicos como TriviaQA o Natural Questions para comparar el rendimiento con otros sistemas RAG publicados

Referencias

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-augmented language model pre-training. *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. *International Conference on Learning Representations (ICLR 2020)*.
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. *Proceedings of the ACL Workshop on Text Summarization Branches Out*, 74–81.